

AUTONOMOUS AGENTS AND MULTI-AGENT SYSTEMS **MULTI-AGENT DEEP LEARNING**

Lukas Esterle | lukas.esterle@ece.au.dk

Mirgita Frasheri | mirgita.frasheri@ece.au.dk

AUTONOMOUS AGENTS

We can do some pre-training of this ...

Multiple heterogenous, autonomous systems

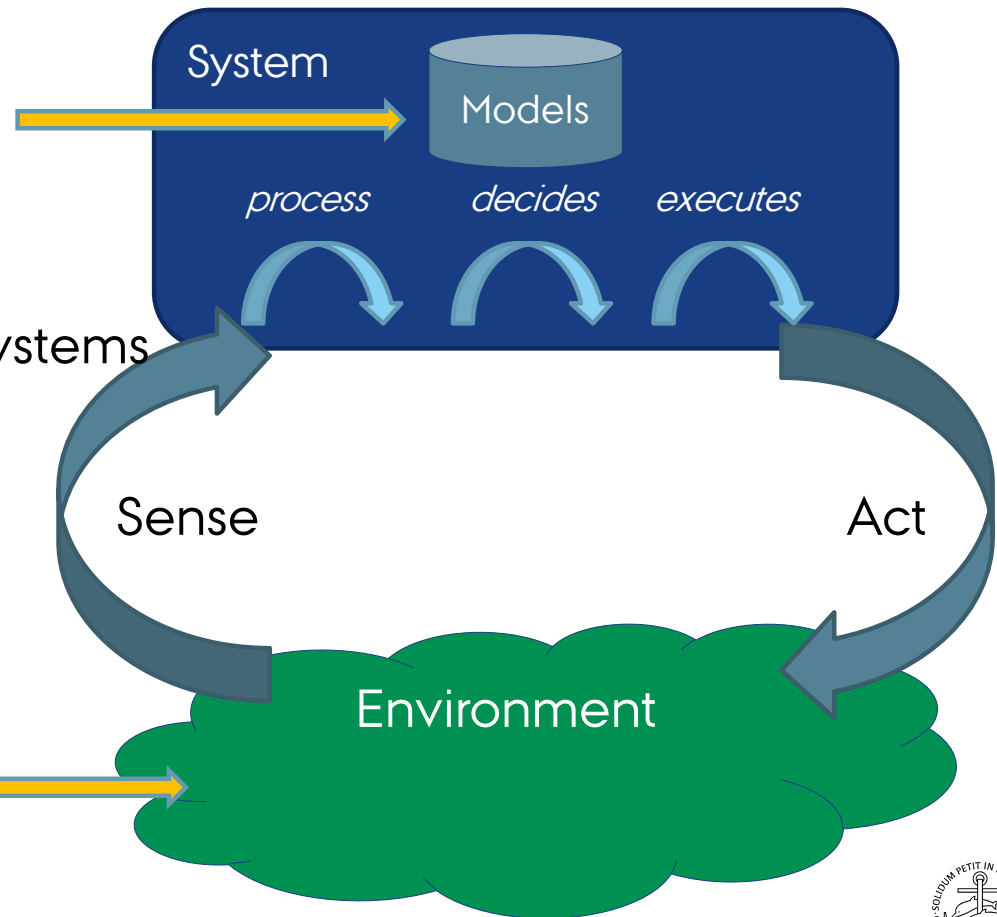
... operating towards goals

... without a central controller

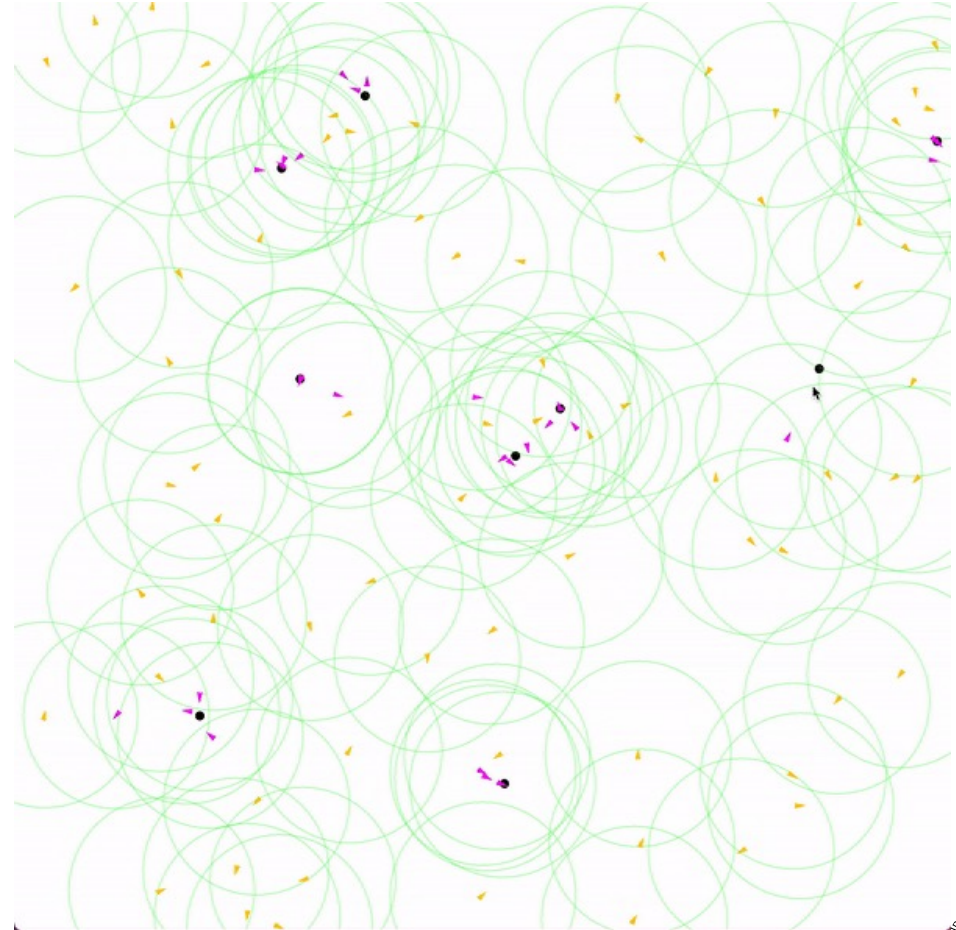
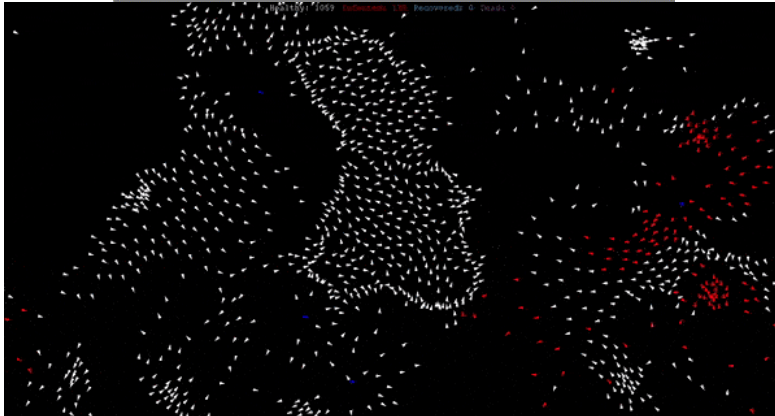
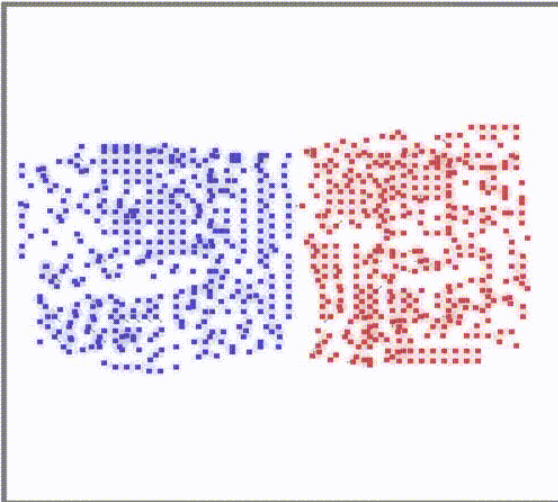
... with limited resources

... based on local information

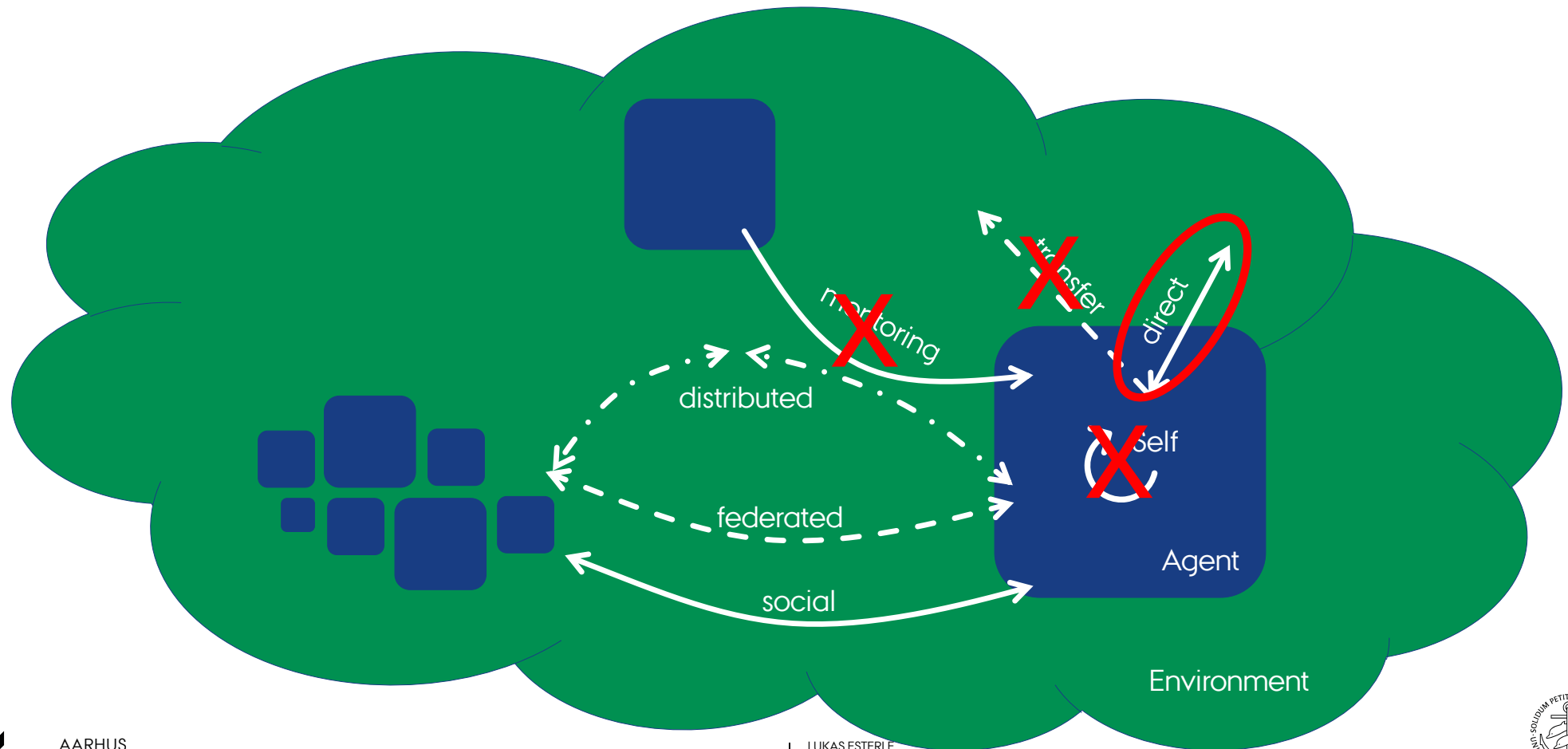
... but this might change!



MULTI-AGENT SYSTEMS

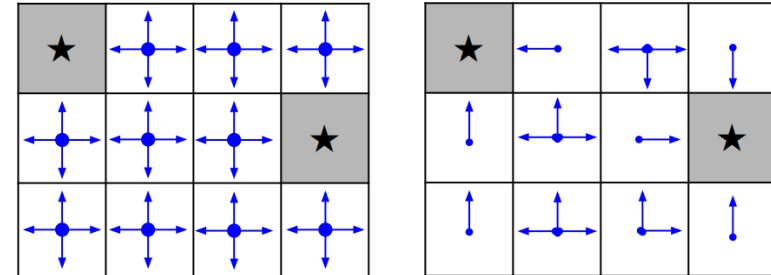


TYPES OF LEARNING IN MAS



Q-LEARNING

Find optimal policies for an agent



The optimal policy π^* corresponds to taking the best action in any state

Learn Q-function – the optimal action-value function maximising the expected cumulative reward achievable from a given (state, action) pair

$$Q^*(s, a) = \max_{\pi} \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t \mid s_0 = s, a_0 = a, \pi \right]$$

Value iteration: Use Bellman equation as iterative update

$$Q_{i+1}(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q_i(s', a') \mid s, a \right]$$

Problem? **Scalability!**

DEEP Q-LEARNING

What's the problem with this?

- Not scalable. Must compute $Q(s, a)$ for every state-action pair. If state is e.g. current game state pixels, computationally infeasible to compute for entire state space!

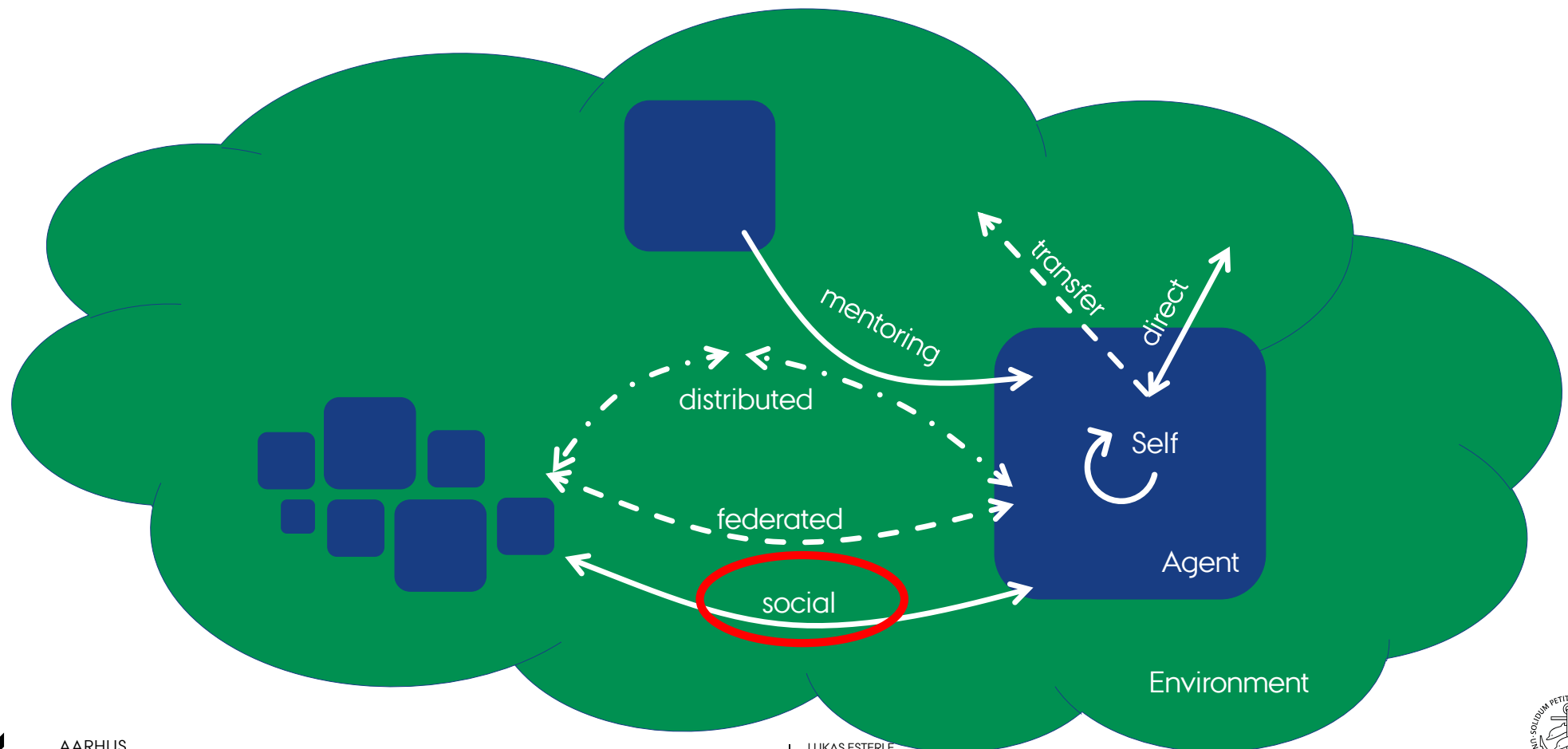
Solution: use a function approximator $Q(s, a; \theta) \approx Q^*(s, a)$. E.g. a neural network!

Function parameters

We can use all kinds of neural networks depending on the requirements.

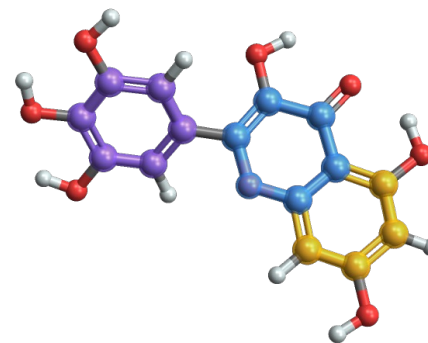
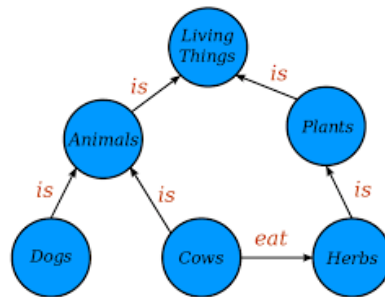
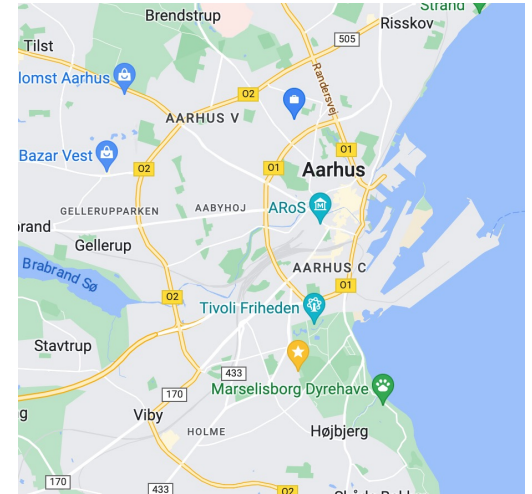
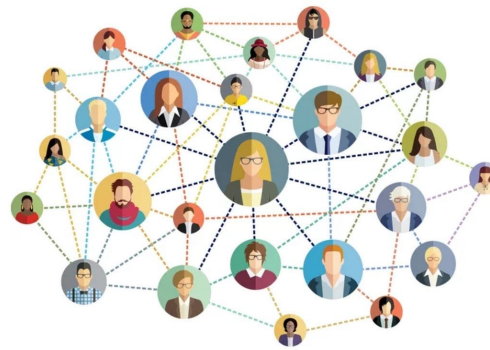
If we want to work with other agents in the environment, Graph Neural Networks are a good way to go.

TYPES OF LEARNING IN MAS

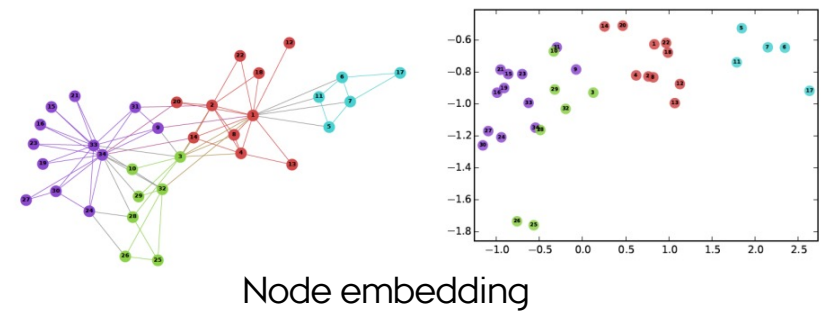
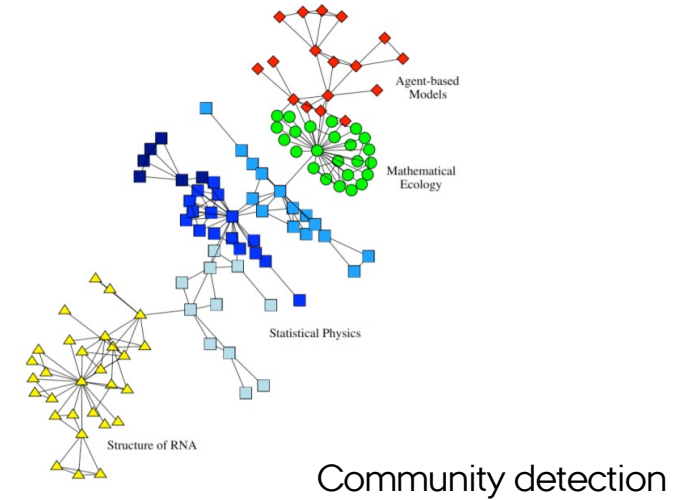
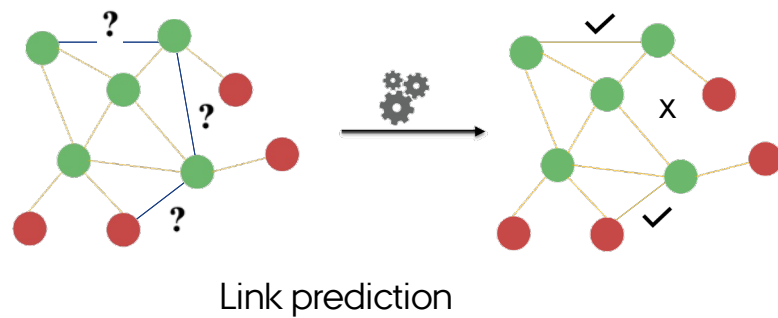
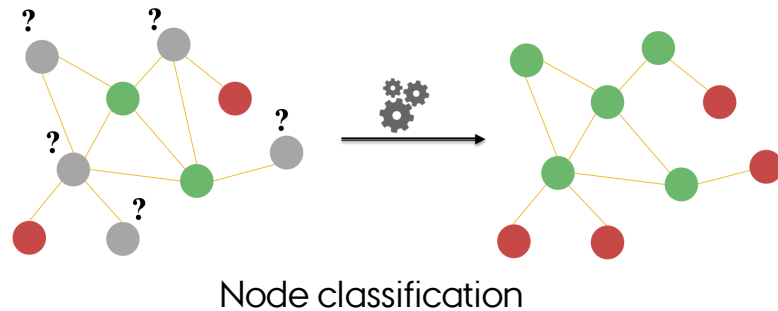


GRAPH NEURAL NETWORKS

Not everything is structured as a grid.



GRAPH NEURAL NETWORKS



GNN FOR AUTONOMOUS AGENTS

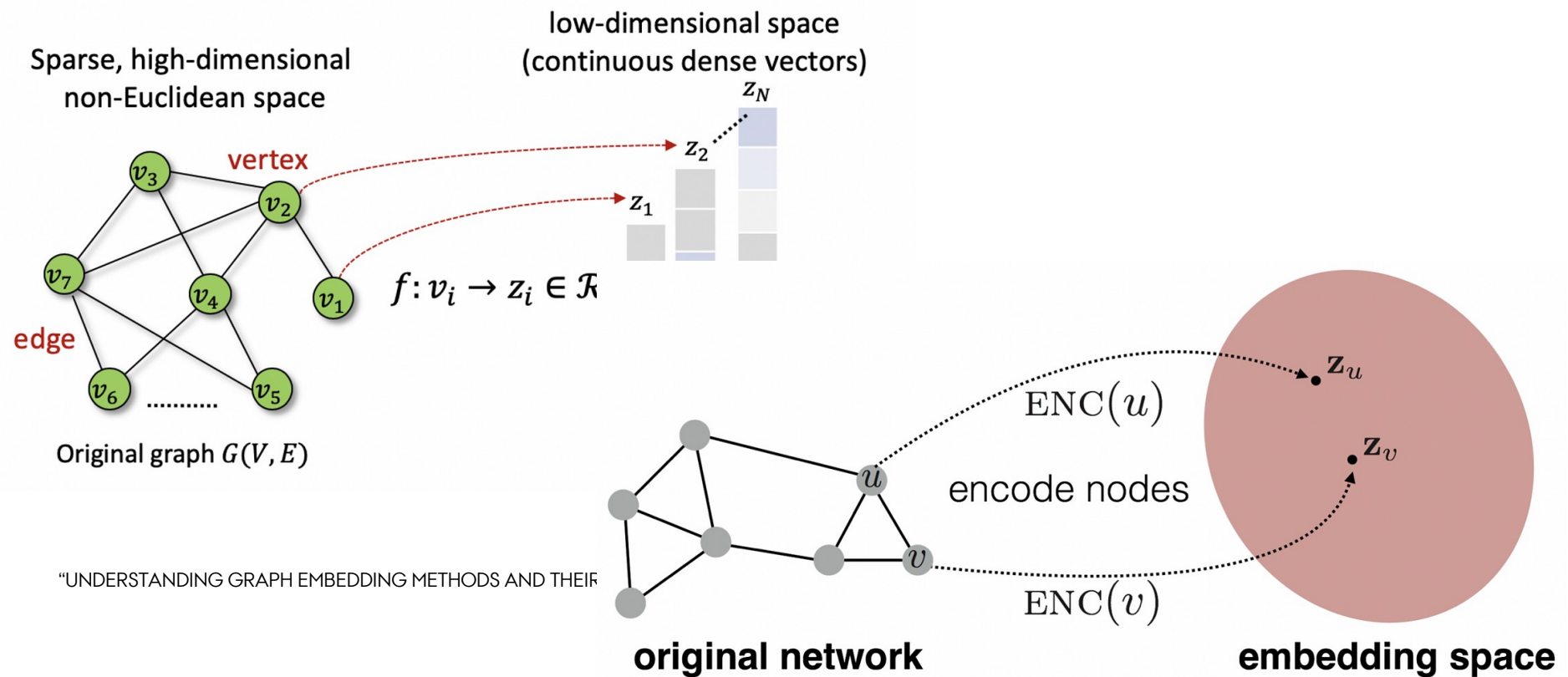
Autonomous agents interact in an environment

Agents in the same environment might have the same experience

Agents interactions can be modelled as social network → graph!

To compare nodes in a graph, we have to learn node embeddings

NODE EMBEDDINGS

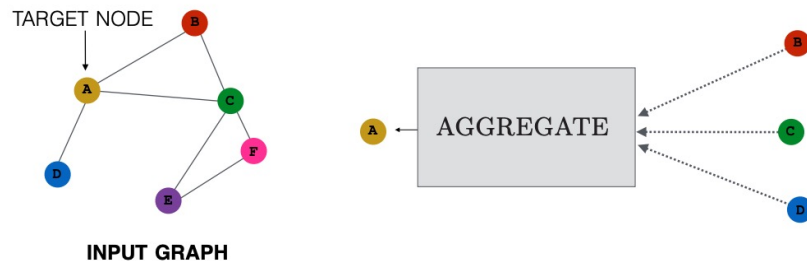


MESSAGE PASSING

At each iteration k

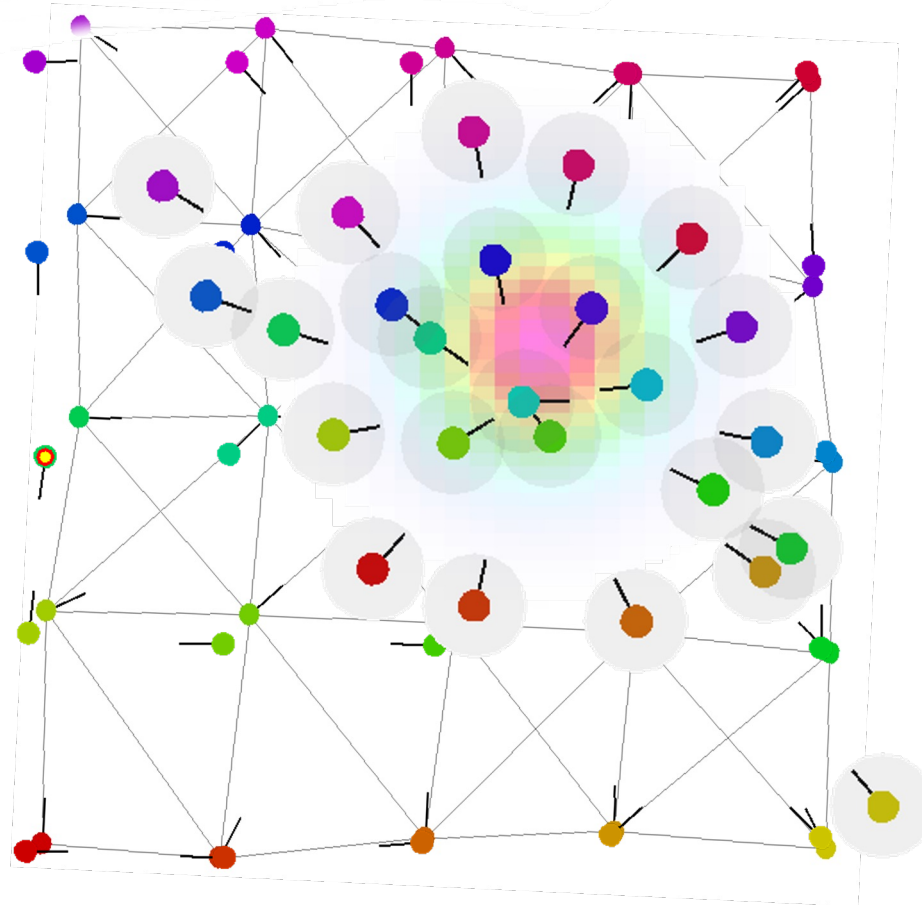
1. **Aggregate** all embeddings from a node u neighbour into a single message $m_{\mathcal{N}(u)}^{(k)}$ based on this information
2. **Update** the embeddings h_u^k of node u by combining information from previous $h_u^{(k-1)}$ embeddings with the new message $m_{\mathcal{N}(u)}^{(k)}$

The more iterations, the more information each node contains in its embeddings

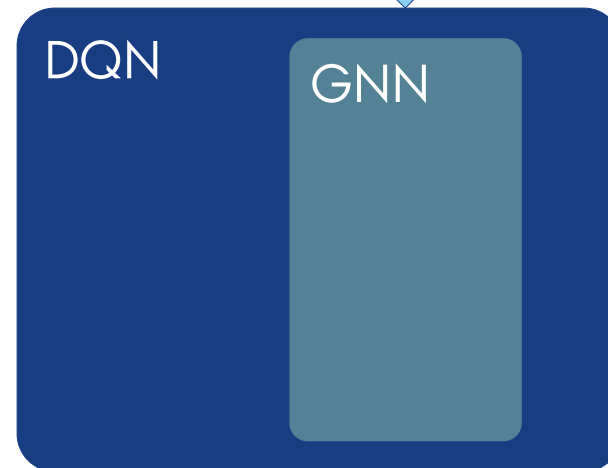
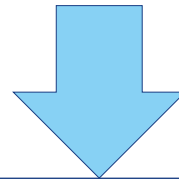
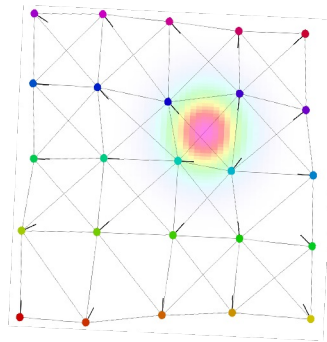


$$\mathbf{h}_u^k = \sigma \left(\mathbf{w}_{self}^k \mathbf{h}_u^{k-1} + \mathbf{w}_{neigh}^k \sum_{v \in \mathcal{N}(u)} \mathbf{h}_v^{k-1} + \mathbf{b}^k \right).$$

MULTI LAYER PERCEPTRON

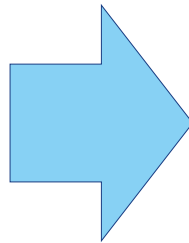
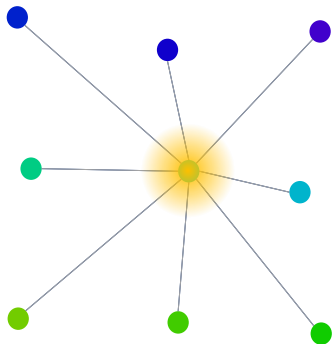


GNN + DQN

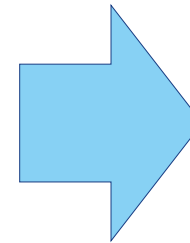
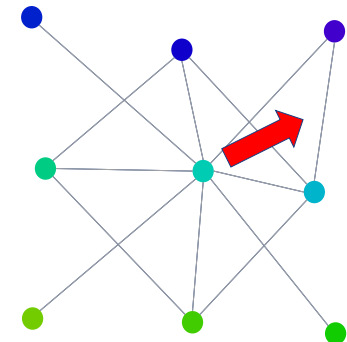


All done locally
on each agent !

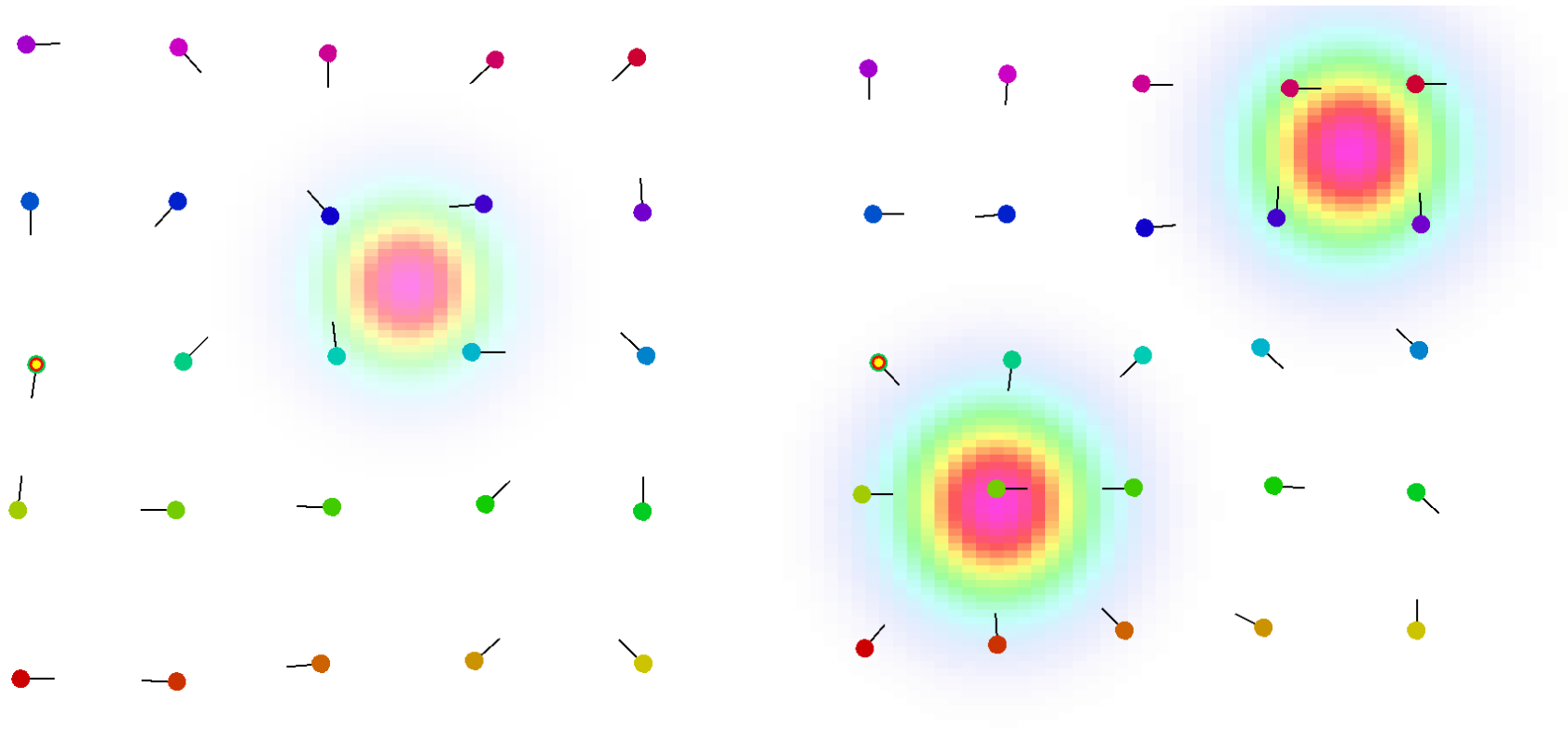
Own perception
+ neighbourhood



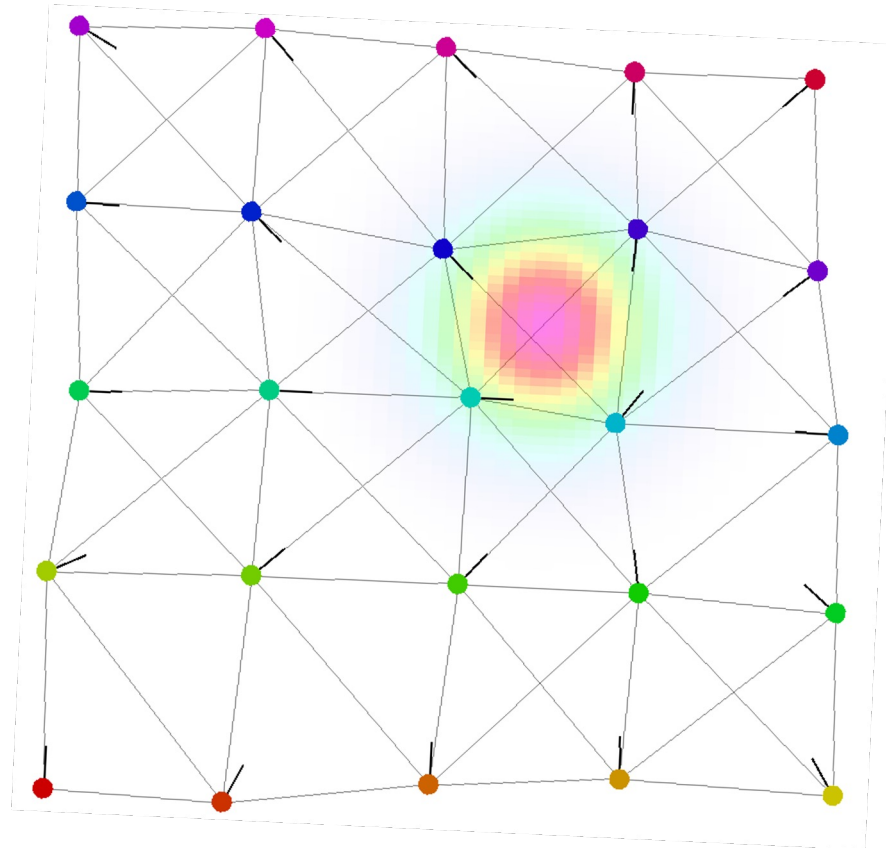
Acceleration vector



GNN + DQN

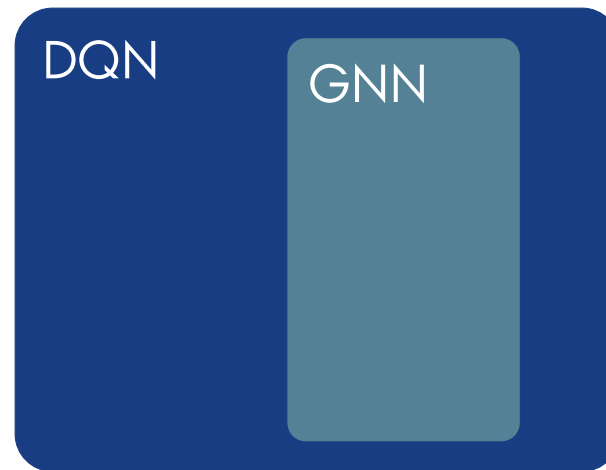
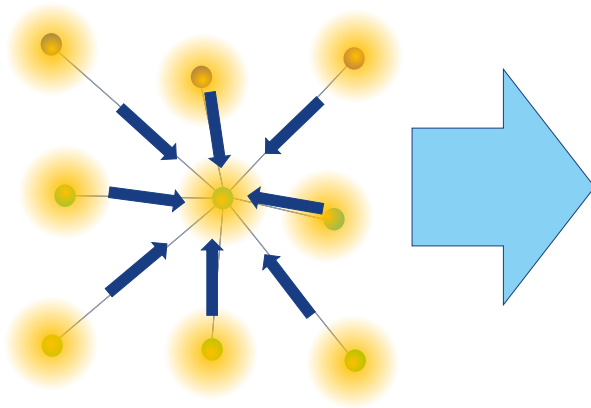


TRAINING

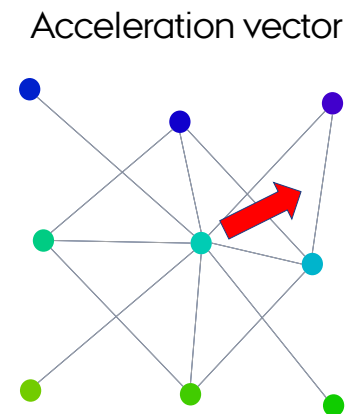


GNN + DQN + AGGREGATE COMPUTING

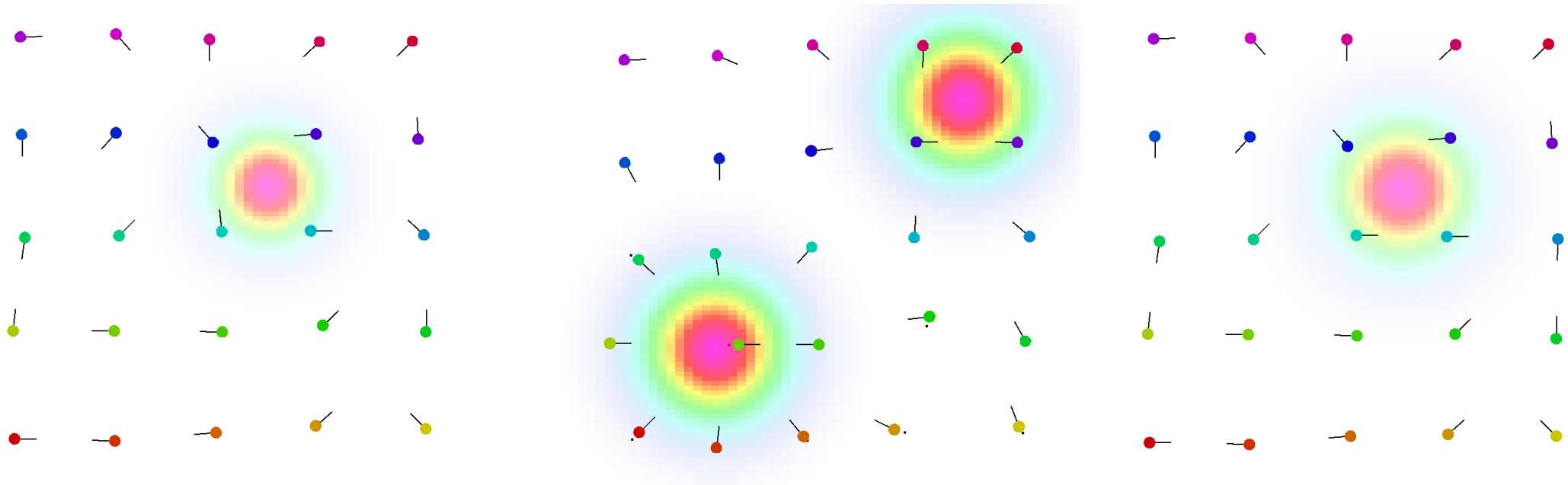
Own perception
+ neighbourhood
+ neighbour perception
(through aggregate computing)



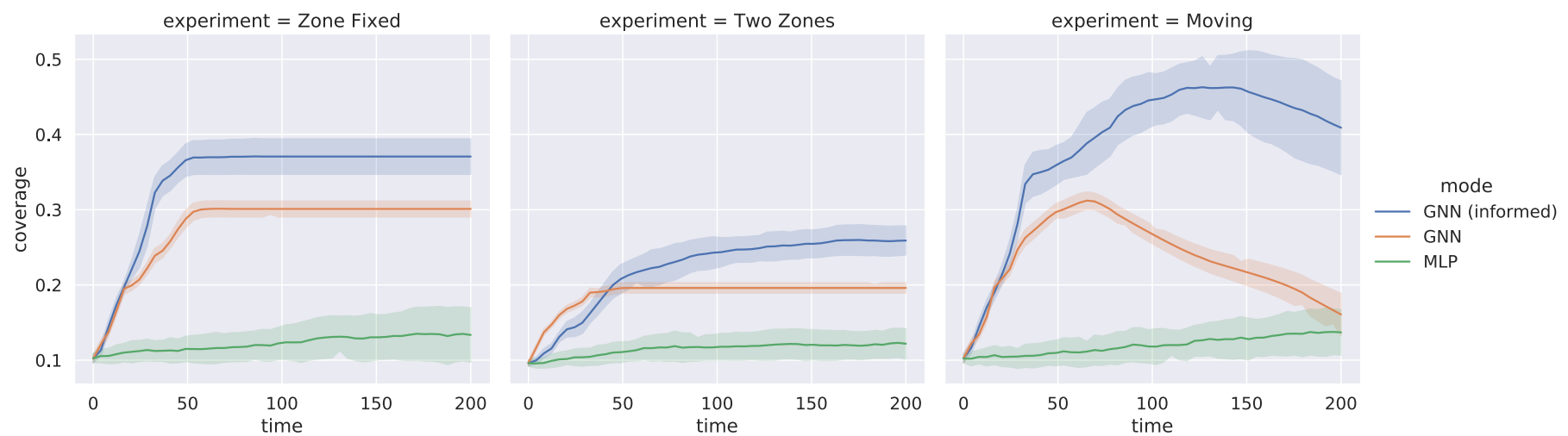
All done locally
on each agent !



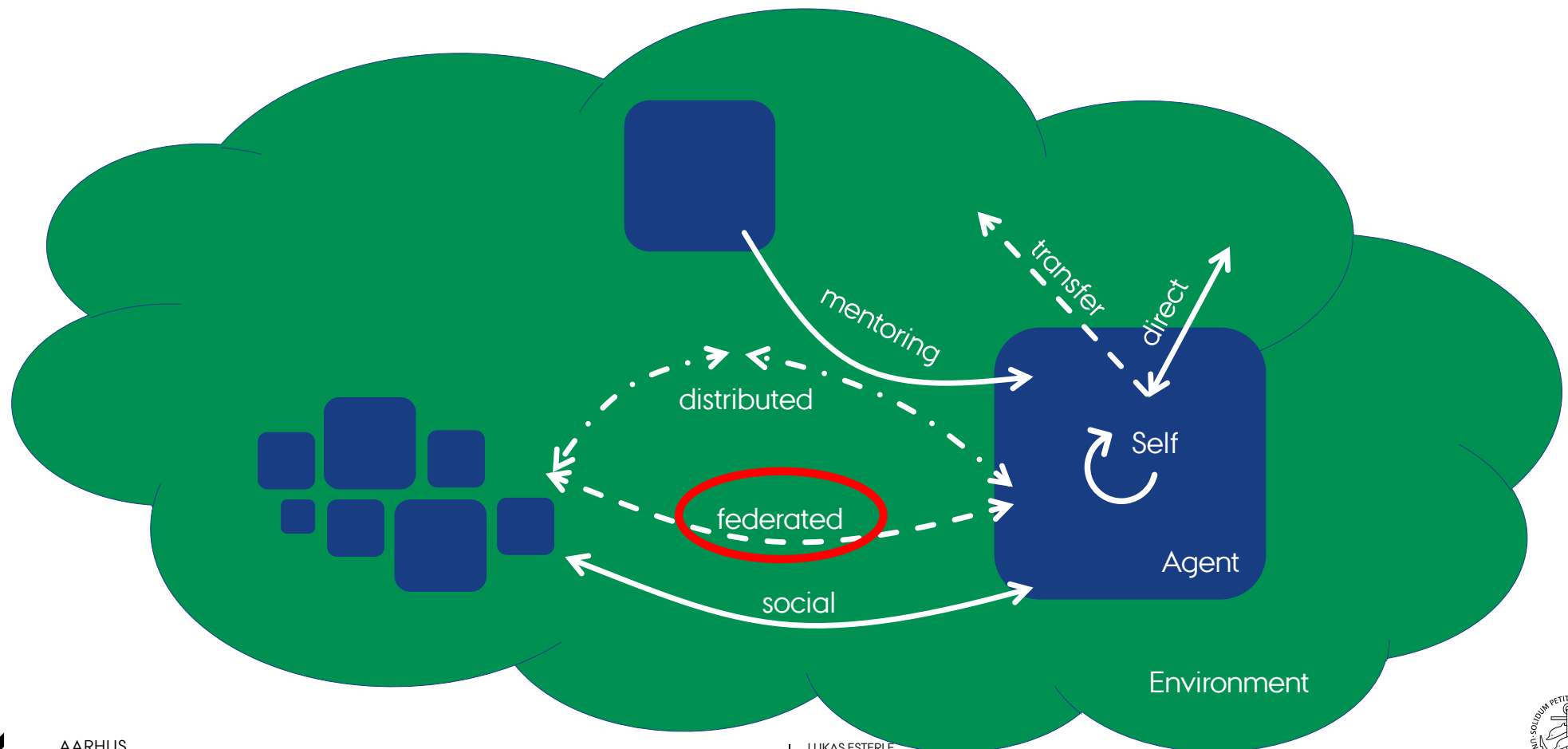
FIELD INFORMED GNN

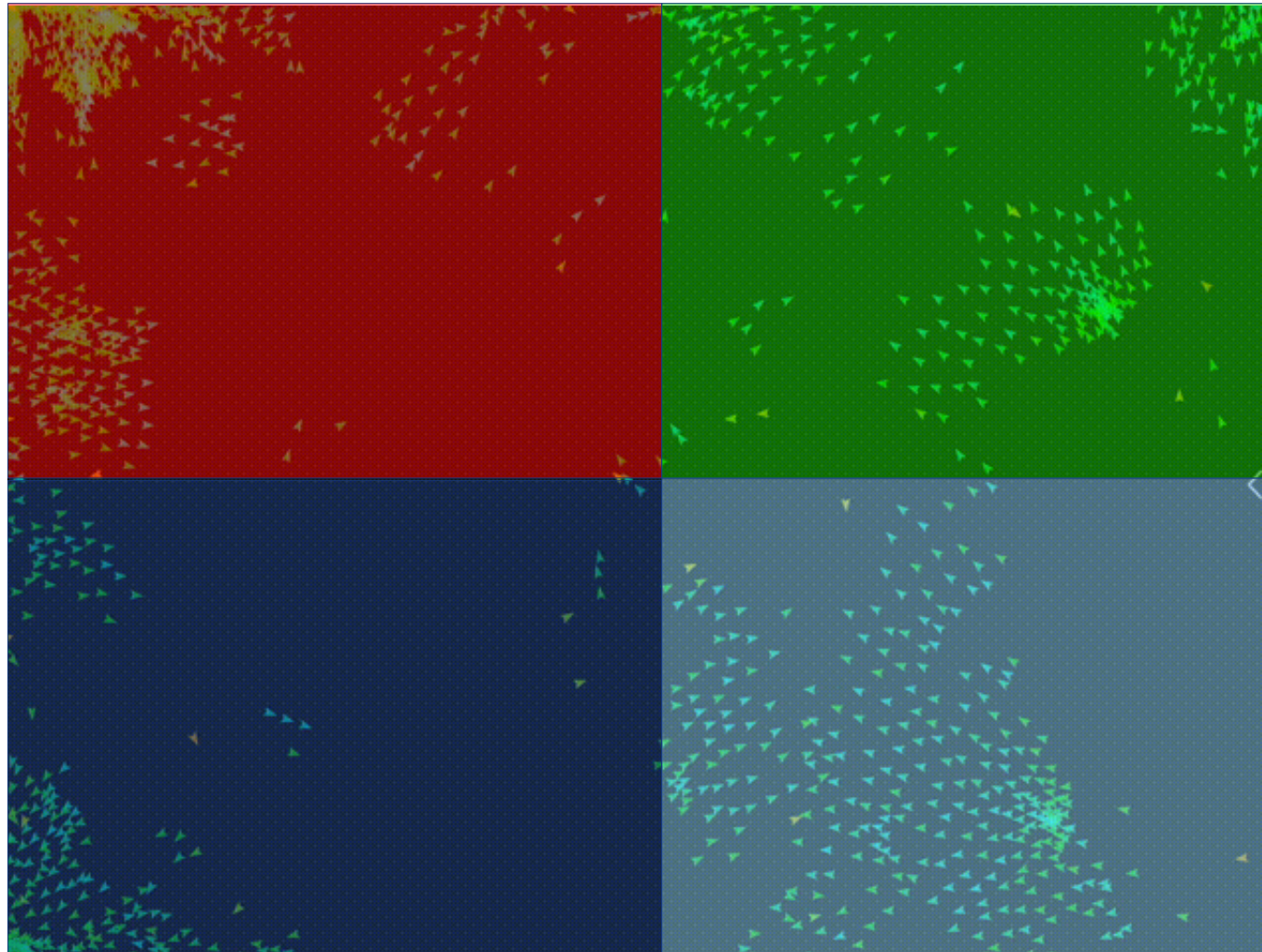


RESULTS COVERAGE

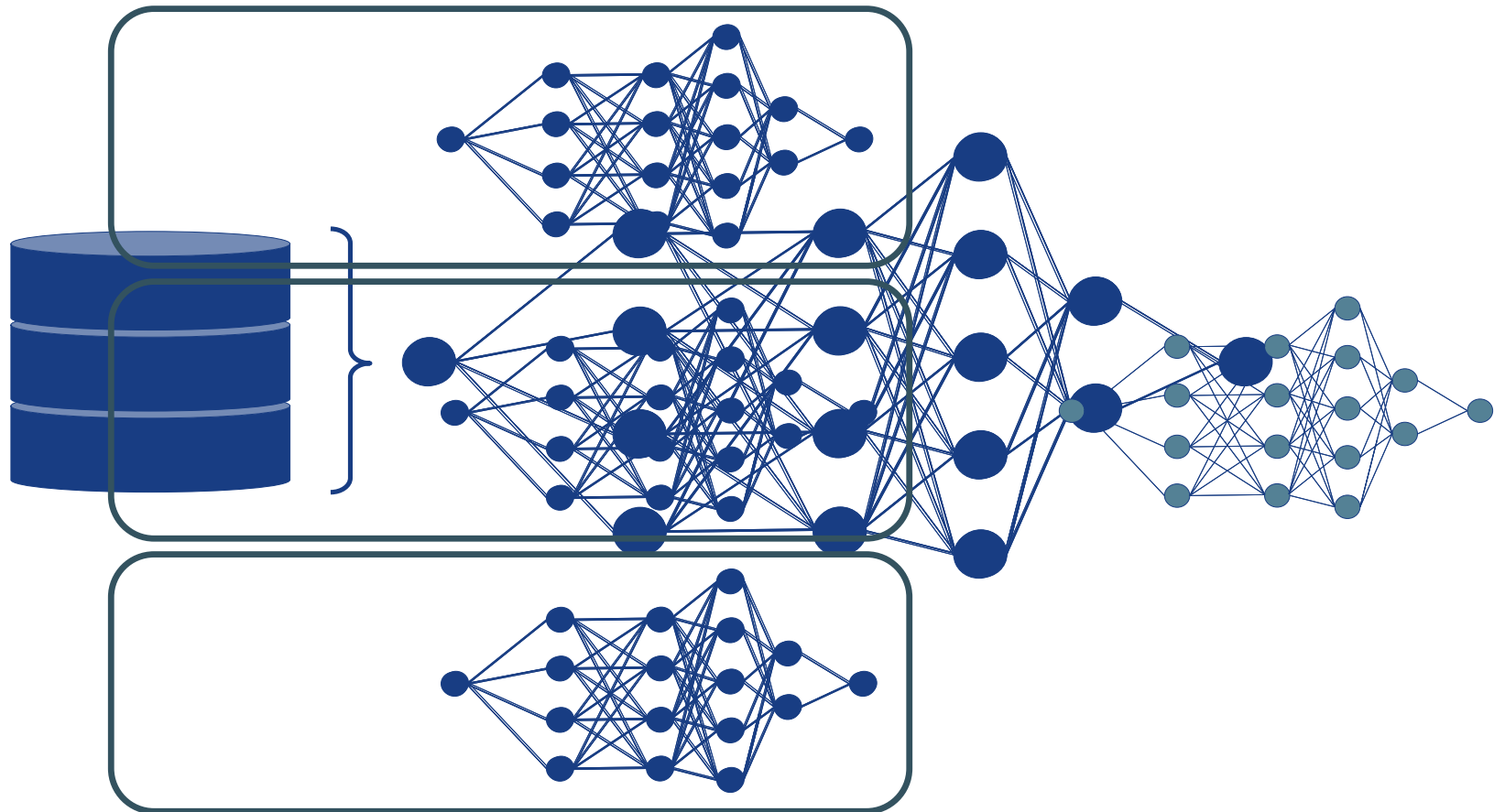


TYPES OF LEARNING IN MAS





FEDERATED LEARNING



Esterle, L. (2022). Deep learning in multiagent systems. In Deep Learning for Robot Perception and Cognition (pp. 435-460). Academic Press.

FEDERATED LEARNING

Let K represent the total number of clients.

Each client k has its own local dataset D_k to train w_k .

The goal is to collaboratively learn a global model w without sharing individual datasets.

Server receives only w_k at regular intervals and performs aggregation step.

FEDERATION PROCESS

Federated Averaging (FedAvg)

Federated Proximal Term (FedProx)

Federated Normalized Averaging (FedNova)

FEDAVG

$$w_{global} = \frac{1}{K} \sum_{k=1}^K w_k$$

Global weights w (i.e., global model) are the average of the local weights w_k (i.e. local model).

FEDERATE PROXIMAL TERM (FEDPROX)

Server select subset S_t of K devices at random (k selected with probability p_k)

Minimize the local objective

$$w_k^{t+1} \approx \arg \min_w h_k(w; w^t) = F_{k(w)} + \frac{\mu}{2} \|w_{global} - w^t\|^2$$

γ_k^t -inexact solution!

The proximal term, $\frac{\mu}{2} \|w - w_{global}\|^2$, penalizes the distance between the local client model w and the global model w_{global} .

FEDERATED NORMALIZED AVERAGING

$$w_{global} = w_{global} + \eta \frac{\tau_k}{\sum_{j=1}^K \tau_j} \Delta w_k$$

FedNova creates updates of weights based proportionally on the number of updates received.

η is the overall update/learning rate and τ is the number of local updates of agent k compared to all agents K .

FEDERATION PROCESS SUMMARY

FedAvg treats everyone equally and assumes the same amount of work in a given time.

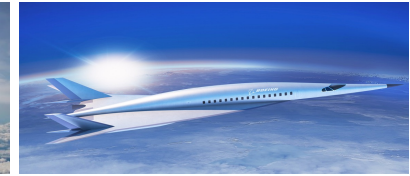
FedProxy keeps everyone close to the global model (client side)

FedNova incorporates updates based on work performed (server side)

Both allow for varying amount of work by the clients

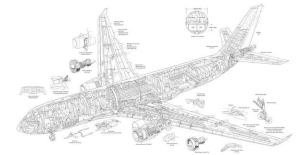
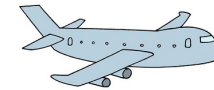
NON-IID DATA

Feature distribution skew (covariate shift)



Label distribution skew (prior probability shift)

Same label, different features (concept shift)
Plane:



Different label, same features (concept shift)



Plane
Airplane
Passenger machine
Flugzeug

Quantity skew or unbalanced-ness

FEDERATED LEARNING – NON-IID

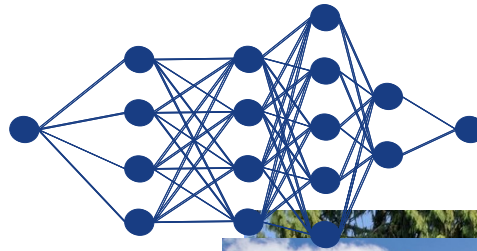
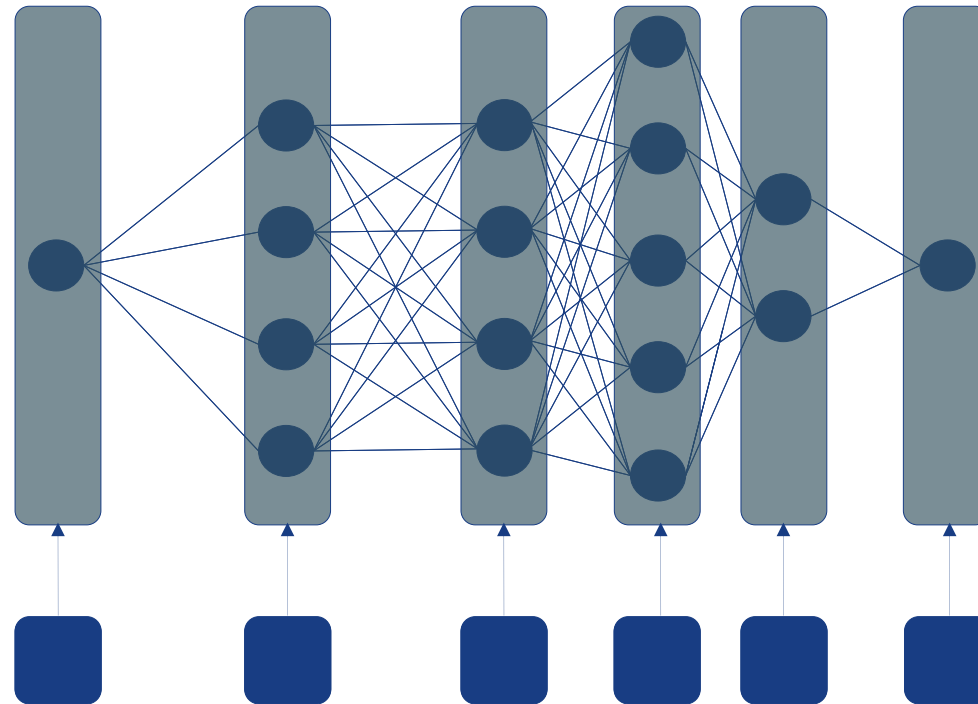


Figure 1. (a) Deep learning in multiagent systems. In Deep Learning for Robot Perception and Cognition (pp. 435-460). Academic Press.

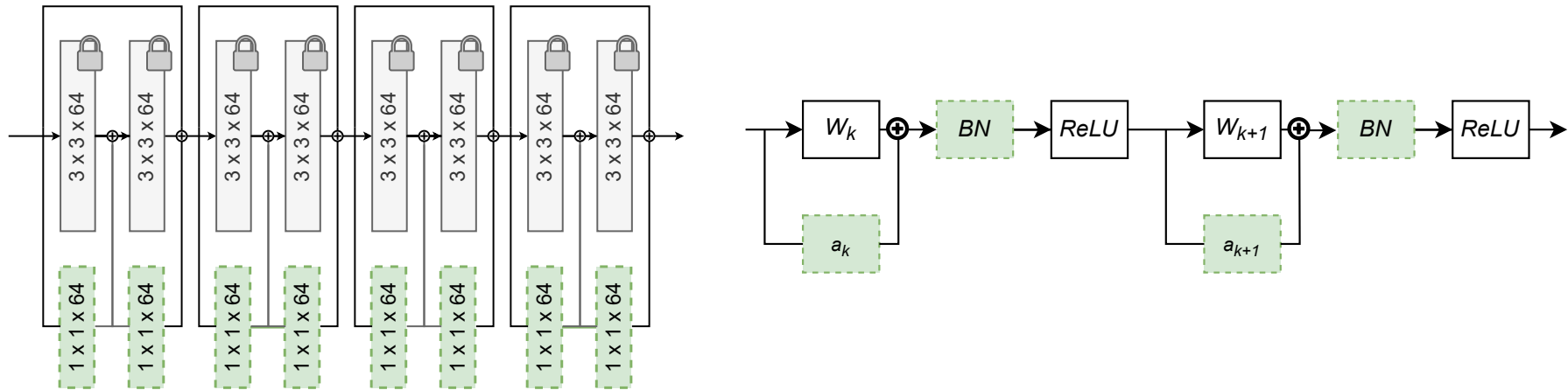


ADAPTIVE PARAMETERIZATION



Elvebakken, M.F., Iosifidis, A. and Esterle, L., 2023. Adaptive Parameterization of Deep Learning Models for Federated Learning. arXiv preprint arXiv:2302.02949.

ADAPTIVE PARAMETERIZATION



Elvebakken, M.F., Iosifidis, A. and Esterle, L., 2023. Adaptive Parameterization of Deep Learning Models for Federated Learning. arXiv preprint arXiv:2302.02949.

FedAvg

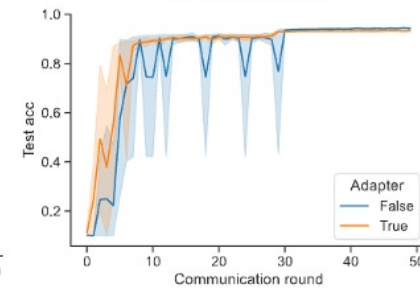
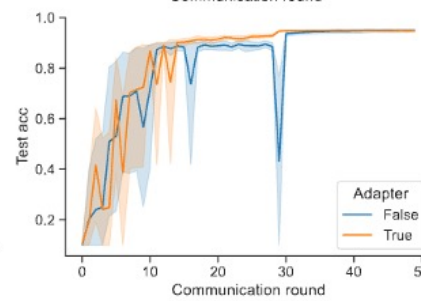
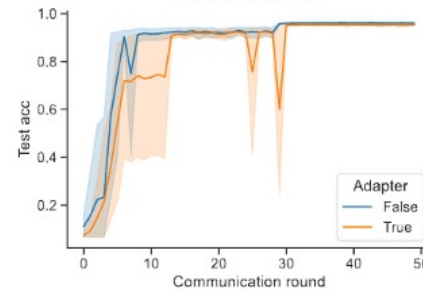
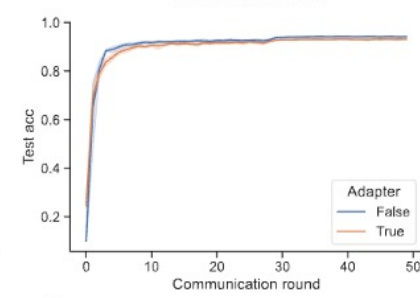
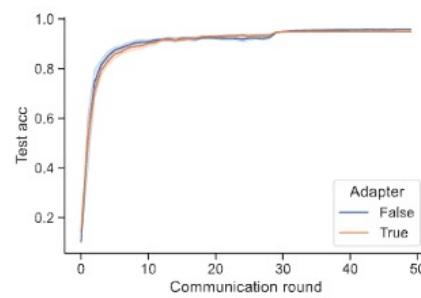
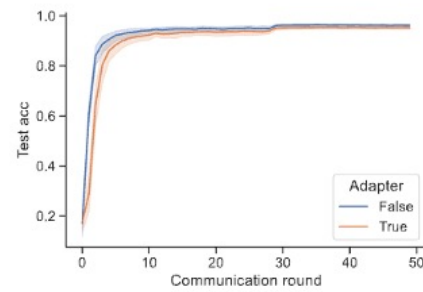
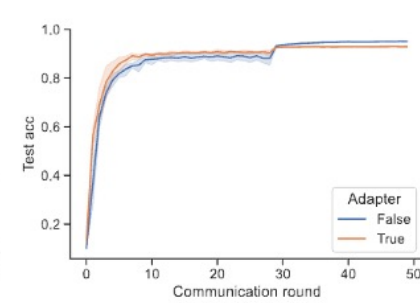
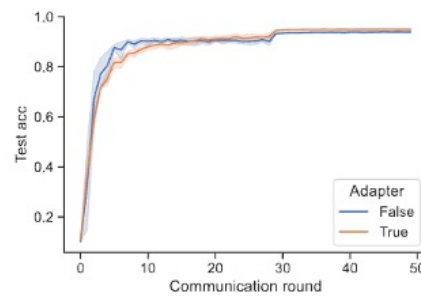
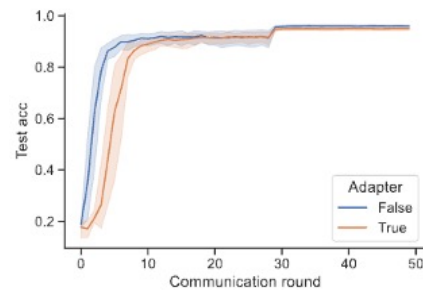
FedProx

FedNova

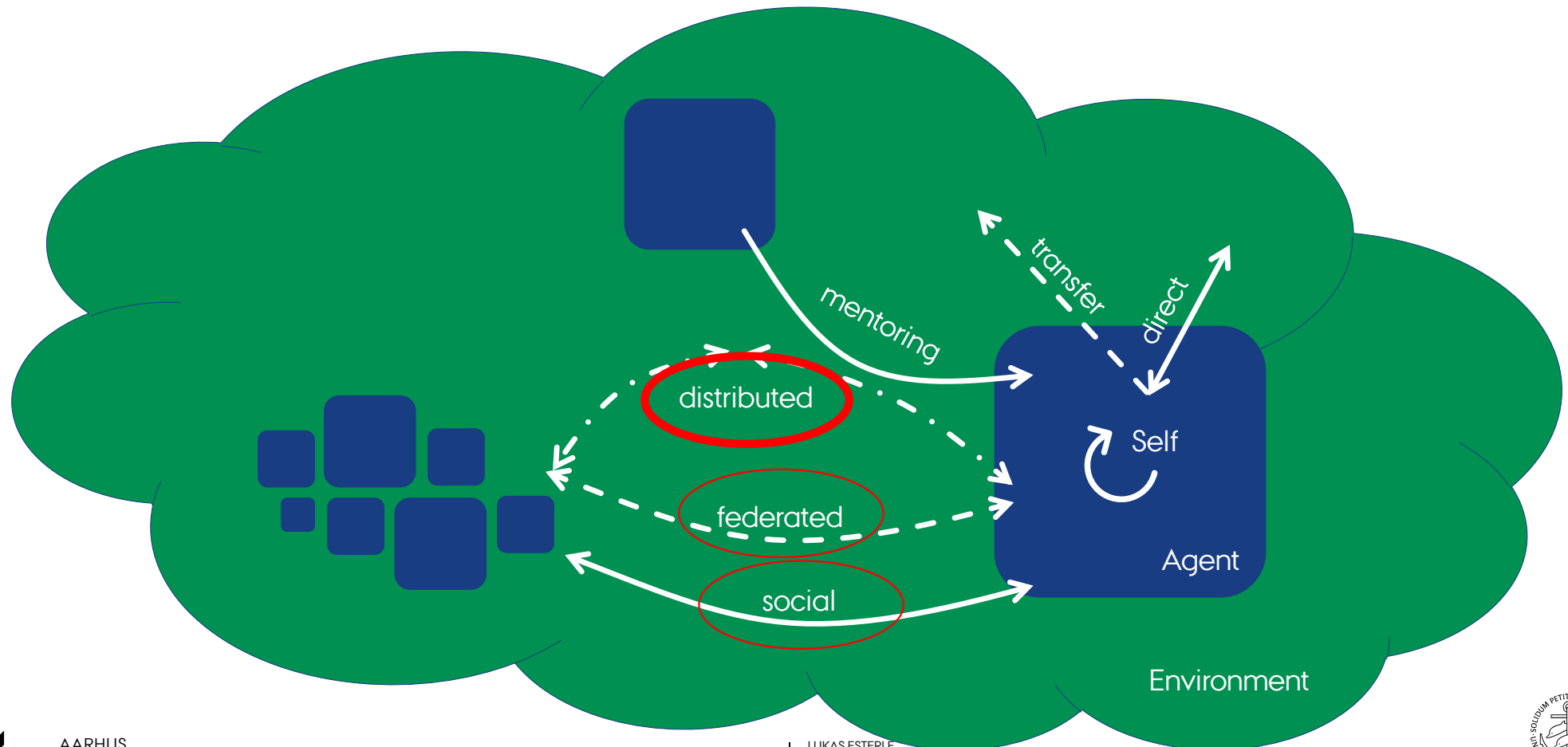
SVHN

Cifar10

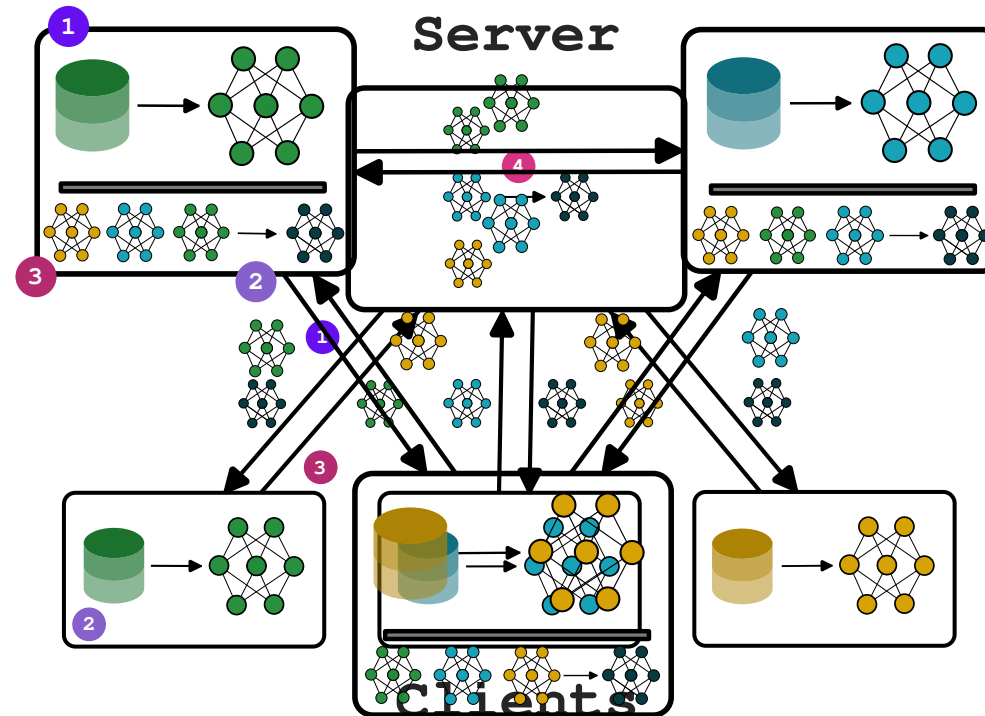
FashionMNIST



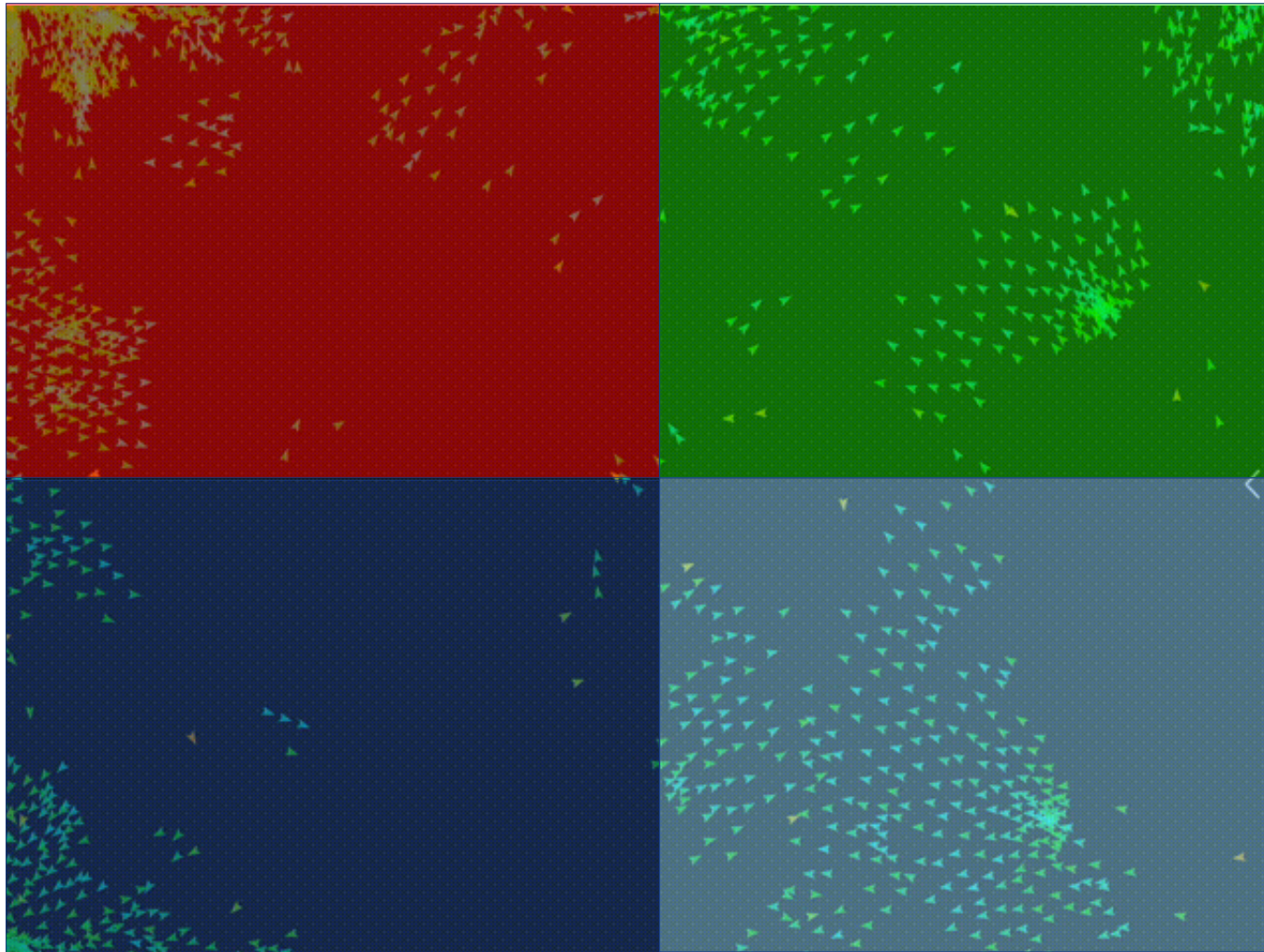
TYPES OF LEARNING IN MAS



FIELD-BASED COORDINATION FOR FEDERATED LEARNING



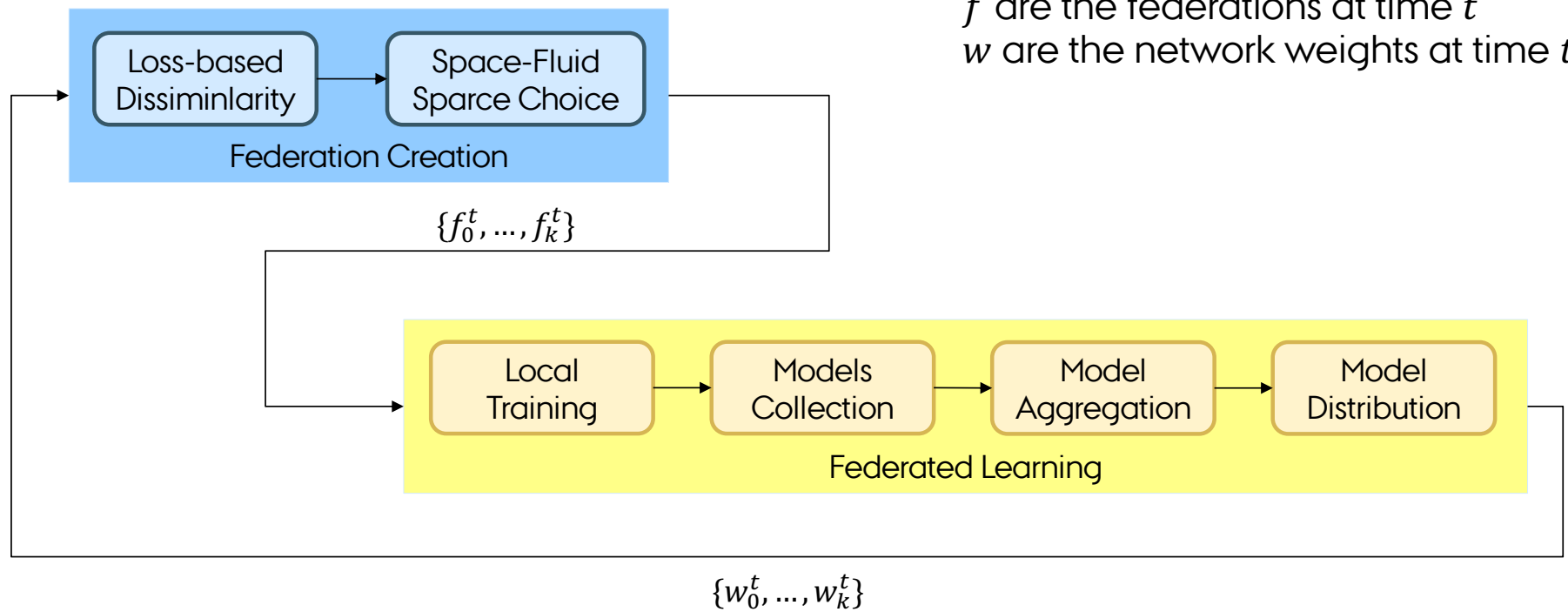
—



PROXIMITY-BASED SELF-FEDERATED LEARNING (PSFL)

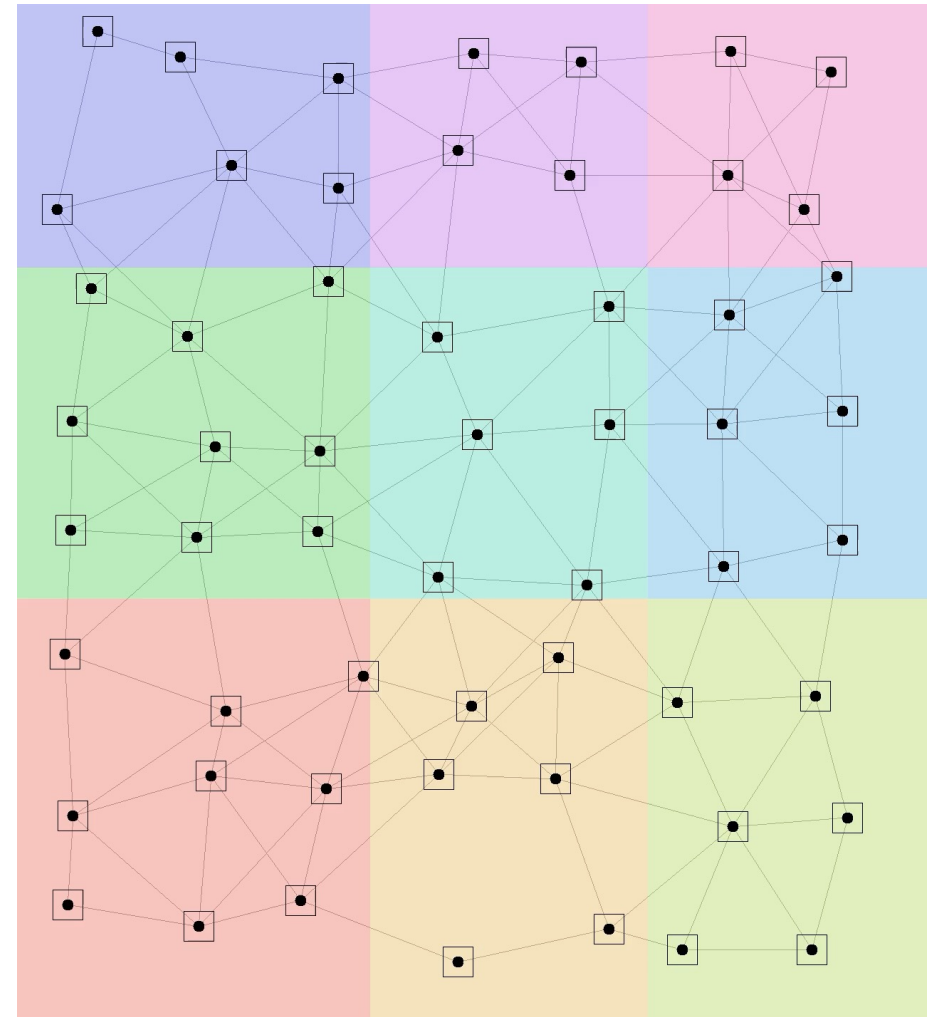
- Enable agents to learn local information
- Use Federated Learning to share models
- Agents in the same area learn similar models
 - Models only need to be shared within their local group

PSFL

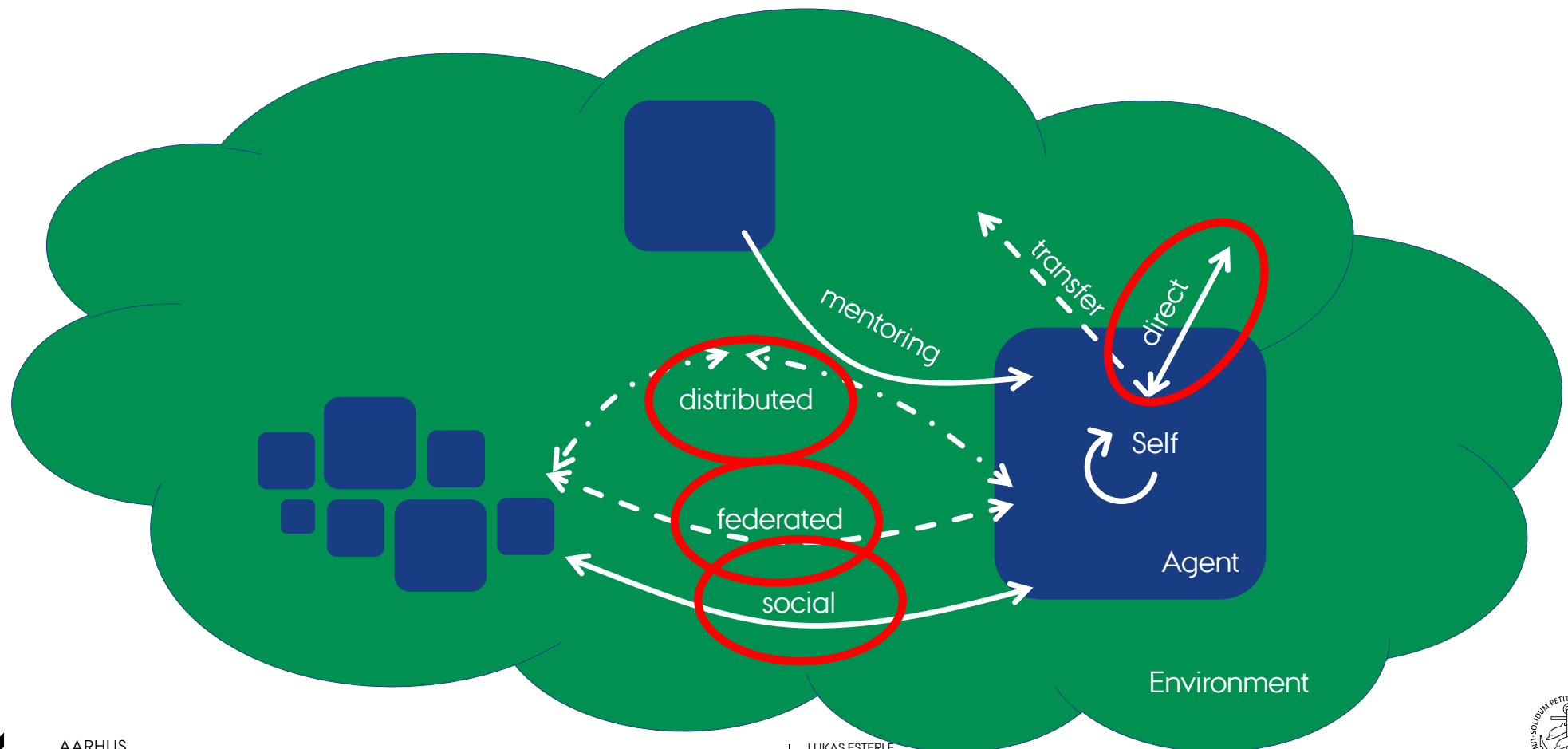


PSFL

- Enable agents to learn local information
- Use Federated Learning to share models
- Agents in the same area learn similar models
 - Models only need to be shared within their local group



TYPES OF LEARNING IN MAS



QUESTIONS (MARL)

Group 1:

Discuss why single agent belief updates are problematic and what to do about them?

What is the Nash equilibrium and the minimax solution and when do we need it?

Group 2:

Discuss the general challenges of MARL and how to tackle them.

What are typical assumptions in MARL and how do they differ from typical game theory assumptions?

Group 3:

Why do we need MARL and why can we not simply use a centralized learning mechanism?

What are agent modelling techniques and how are they used in MARL?

Group 4:

Discuss the problem of joint action learning and how to overcome it.

What is policy learning, how does it work and how can we improve it in MARL?